

PEA Code and Run Guide

Status and prerequisites

This is an isolated filtered copy of current working files, including uncommitted changes and untracked datasets. Backend HEAD is 0a31252993b9629739ff7ee625d878f67ac6880c; frontend HEAD is ed3ec28ccd09e9ee21cfc3455e96876bceb00225. File hashes, not HEAD alone, identify this release. No source project services were stopped.

Fresh dependency installation, pytest, frontend test/build, import checks and copied-service startup were NOT RUN in this accelerated delivery. The commands are verified against source entry points, not claimed tested on a fresh machine. Do not mark the package fully runnable until the pending checks pass.

pea-backend/pyproject.toml requires Python ≥ 3.11 ; pea-frontend/package.json requires Node $\geq 22.18.0$ and npm. Do not reuse a copied .venv or node_modules. Incompatible pydantic_core binaries can cause the earlier import failure.

Install and configure

```
cd .\pea-backend
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
Copy-Item .env.example .env
.\.venv\Scripts\python.exe -m app hash-password
```

hash-password prompts without echoing, confirms the password and prints a hash. Configure a new private AUTH_ACCOUNTS entry with username, password_hash, display_name, principal_id, employee_ids and permitted actions. Empty account examples deliberately provide no usable login. Do not weaken authentication. No real password or API key is supplied. Optional ARK_API_KEY and OPENROUTER_API_KEY belong only in the backend private environment; never put secrets in VITE_ fields.

```
cd ../pea-frontend
npm ci
Copy-Item .env.example .env
```

Run npm in pea-frontend, where package.json exists. API keys are unnecessary for Rule-based auditing. Model operations incur cost only after configuration and an explicit user action. No past call budget is reused.

Writable application startup

In a backend terminal:

```
cd pea-backend
.\.venv\Scripts\python.exe -m app serve
```

In a second terminal from the release root:

```
cd pea-frontend
npm run dev
```

Defaults are backend 127.0.0.1:8000 and frontend 127.0.0.1:5173. If occupied, choose unused PORT and Vite --port values, and align API_PROXY_TARGET. Do not kill unrelated processes. vite.config.ts actually reads API_PROXY_TARGET for /v1, defaulting to port 8000. Empty VITE_API_BASE_URL uses same-origin requests; a separate origin requires matching backend CORS. npm run preview does not provide the development API proxy.

Check /health, /docs, login and a synthetic Rule-based audit. Health alone does not validate rubric loading. Stop only your own terminals with Ctrl+C. FileStore allows one process per runtime directory; do not share active storage between server and evaluation CLI.

Rubric and profile resolution

Settings reads .env from the backend module-derived root; environment variables override it. app/rubric.py requires both registration and settings.rubric_versions membership. v1.0 loads from app/rubrics/v1.0.json relative to the loader. v0.4 remains an in-code dictionary; no v0.4.json is required.

The safe release example allows v0.4 and v1.0; the frontend requests v1.0. The approved artifact is copied unchanged. No validation is bypassed. The historical RUBRIC_NOT_FOUND repair is not claimed reverified in this copied service. An actual copied-service rubric load remains pending.

Current Agent planning source retains missing-fact-query-2. The historical tiny profile is not equivalent to agent-demo-v2-validated; consult app/agent_demo_v2.py, app/agent_validation.py and executed manifests before deliberately enabling live Agent use. Do not invoke old multi-case validation scripts just to view saved results.

The old 5176/8003 read-only viewer and 5177/8004 recording service are separate from the writable product. This release does not start or certify either historical viewer, copy active authentication state, or label one as writable. JSON/CSV reports are directly readable offline.

Module responsibilities

Module	Responsibility
app/main.py and cli.py	API lifecycle and routes; serve, hash-password, evaluate entry points
app/config.py, auth.py, passwords.py	Strict settings, accounts, scopes and sessions
app/schemas.py and rubric.py	Input and Assessment validation; rubric registration/loading
app/audits.py and models.py	Initial/child audits, provider transport, budgets and validation
app/agent.py	Bounded planning, duplicate/empty-result stopping and evidence merge
app/evidence.py and hr_search.py	Authorized local source/date filtering and keyword matching
app/evaluation.py and dataset_inputs.py	Input adaptation, persisted predictions, separate Gold and metrics
app/storage.py and responses.py	Atomic local JSON records and response projection
frontend src/pages and components	Forms, assessments, human review, follow-up and report UI
frontend src/api, stores and router	Requests, sessions and navigation guards

Offline checks still required

backend, with isolated storage and blocked model transport

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe data\validate_data.py
.\.venv\Scripts\python.exe -m compileall -q app
```

frontend

```
npm test
```

```
npm run build
```

Inspect fixtures first, use mock credentials and a new temporary runtime, and prevent production .env loading. Browser tests require their Playwright dependencies. These tests/builds were not newly run. The exact-query offline script scripts/replay_agent002_queries.py must have its source/output paths reviewed before use. python -m app evaluate PROFILE --principal PRINCIPAL executes an evaluation and may dispatch paid calls; it is not an offline arithmetic command.